

Módulo 2

Programación Orientada a Objetos en C#



2. Herencia y polimorfismo

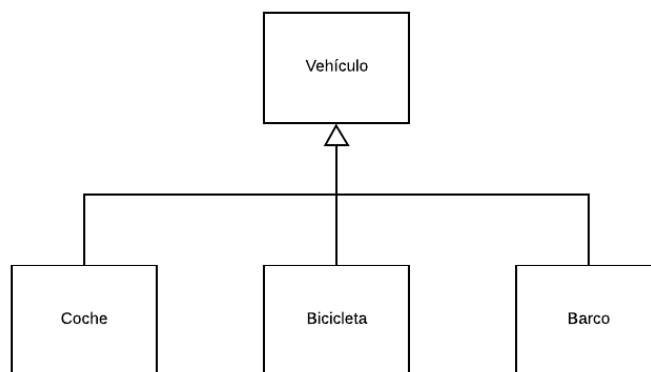
Curso de Iniciación a la programación con C#

Index

Herencia.....	3
Clases derivadas.....	3
Sobrescritura de métodos heredados.....	4
Llamar al constructor o métodos de la clase base.....	5
Clase Object.....	6
Método ToString.....	6
Método Equals.....	7
Clonar objetos.....	8
Clases abstractas.....	9
Interfaces.....	10
Polimorfismo.....	11
Herencia con records.....	15
Métodos de la clase Object autogenerados.....	15
Clonación Inmutable (La expresión with).....	15

Herencia

La herencia permite crear clases derivadas (o clases hijas) a partir de otras (clases base o padre), que heredan su comportamiento y lo extienden más allá. Esto también se conoce como especialización. Por ejemplo, podríamos tener la clase Vehículo con todas las propiedades comunes a un vehículo (terreno por el que se mueve, si tiene motor o no, ruedas, ...) y otras más específicas como **Coche**, **Barco** o **Bicicleta** que además de todo lo anterior tuvieran información y comportamientos específicos a su tipo de vehículo como pueden ser los metros de eslora (Barco), tipo de combustible (Coche), tipo de bicicleta (montaña, carretera, híbrida, ...). La herencia se representa gráficamente así:



Existen lenguajes de programación como C++ que permiten herencia múltiple, es decir, una clase puede heredar las características de 2 o más clases. Sin embargo, lenguajes como C# o Java, entre muchos otros, sólo permiten la herencia simple (de una clase padre). Existen formas de heredar características de más de una entidad diferente si en lugar de clases heredamos de interfaces, pero eso es algo que veremos más adelante.

Clases derivadas

Creamos una clase derivada indicando detrás del nombre de la misma, el nombre de la clase base separado por 2 puntos:

```
class Coche : Vehiculo {...}
```

Las clases derivadas heredan los métodos y las propiedades de la clase base, se incluyen sin tener que volverlos a declarar. Además de tener los suyos propios.

```
class Animal(string nombre, double peso)
{
    public double Peso { get; protected set; } = peso;
    public string Nombre { get; init; } = nombre;

    public void Comer()
    {
        Peso += new Random().NextDouble() * 0.5;
        System.Console.WriteLine($"Nam ñam. Ahora peso {Peso:F2} kilos");
    }
}
```

```

class Ave(string nombre, bool puedeVolar) : Animal(peso, nombre)
{
    public bool PuedeVolar { get; init; } = puedeVolar;

    public void PonerHuevos()
    {
        int numHuevos = new Random().Next(1, 6);
        System.Console.WriteLine($"He puesto {numHuevos} huevos");
    }
}

Ave pajaro = new Ave(true, 1, "Loro");
pajaro.Comer();
pajaro.PonerHuevos();

```

Ñam ñam. Ahora peso 1,49 kilos
 He puesto 5 huevos

Es importante aclarar que aunque hereda TODAS las propiedades y métodos de la clase base, una clase derivada no podrá acceder o usar directamente aquello declarados como **privado**. Entonces, ¿cómo podremos acceder o modificar un atributo privado desde una clase derivada?. Pues utilizando propiedades o métodos **públicos** o **protected**.

```

class Empleado(int rangoInicial)
{
    // atributo estático de solo lectura (valor compartido por todos los objetos)
    private static readonly string[] nombresRango = ["Junior", "Estándar", "Senior", "Experto", "Pro"];
    private int Rango { get; set => field = Math.Clamp(value, 1, 5); } = rangoInicial;
    public string NombreRango { get => nombresRango[Rango - 1]; }

    public void Ascender()
    {
        Rango++;
        Console.WriteLine($"Empleado ascendido al rango { Rango }");
    }
}

class Gerente(int rangoInicial) : Empleado(rangoInicial);

var gerente = new Gerente(3);
gerente.Ascender(); // Método público que accede a la propiedad privada
Console.WriteLine(gerente.NombreRango); // Propiedad pública (también accede)

Empleado ascendido al rango 4
Experto

```

Existe la posibilidad de declarar un método o atributo en la clase base como **protected**. Este modificador es equivalente a **private** con la diferencia de que las clases derivadas sí que podrán acceder directamente a él.

Sobrescritura de métodos heredados

A veces necesitaremos que algunos métodos heredados tengan un comportamiento diferente al definido en la clase base. Para ello, en la clase base debemos añadir el modificador **virtual** al método que queramos sobrescribir en clases derivadas, y el modificador **override** en el nuevo método de la clase derivada que sustituya a su equivalente en la clase base.

```

public class Animal(string nombre, double peso)
{
    ...

    public virtual void Comer()
    {
        Peso += new Random().NextDouble() * 0.5;
        System.Console.WriteLine($"Ñam ñam. Ahora peso {peso:F2} kilos");
    }
}

class Ave(string nombre, double peso, bool puedeVolar) : Animal(nombre, peso)
{
    ...

    public override void Comer()
    {
        // Definimos que un ave aumenta su peso un 5% siempre al comer
        Peso *= 1.05;
        System.Console.WriteLine($"Pio pio. He comido y ahora peso {Peso}
kilos");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Animal animal = new Animal();
        animal.Comer();
        Ave pajaro = new Ave(true);
        pajaro.Comer();
    }
}

```

Ñam ñam. Ahora peso 1,32 kilos
Pio pio. He comido y ahora peso 1,05 kilos

Llamar al constructor o métodos de la clase base

Cuando creamos un objeto de una clase derivada a partir de su constructor, también se llama implícitamente al constructor de la clase base primero. Es decir, se ejecutan ambos constructores. En el caso de que el constructor de la clase base necesite recibir algún parámetro, bien porque no exista un constructor por defecto (sin parámetros) o porque queramos utilizar un constructor parametrizado para inicializar las propiedades, debemos indicarle dichos parámetros y pasárselos al constructor de la clase base.

Si la clase derivada tiene un **constructor primario**, podemos llamar al constructor de la clase base directamente junto al nombre de su clase.

```

class Ave(string nombre, double peso, bool puedeVolar) : Animal(peso, nombre)

```

En el caso de un constructor normal, esto se hace utilizando la palabra reservada **base**, que hace referencia a la clase base.

```

public class Animal
{
    ...
    public Animal(string nombre, double peso)
    {
        Nombre = nombre;
    }
}

```

```

        Peso = peso;
    }
    ...
}

public class Ave : Animal
{
    public Ave(string nombre, double peso, bool puedeVolar)
        : base(nombre, peso)
    {
        PuedeVolar = puedeVolar;
    }
    ...
}

```

Cuando sobrescribimos un método de la clase base, de manera similar a lo que ocurre con el constructor, podemos seguir llamándolo y ejecutar su código. A veces esto simplemente se hace porque queremos realizar lo mismo que se hacía en la clase base más una serie de acciones añadidas. Esto también se hace utilizando la palabra **base** para referirnos a la implementación de la clase base (de forma similar a como usamos `this` si queremos referenciar al objeto actual).

```

class Mamifero(string nombre, double peso, bool carnivoros) : Animal(nombre,
peso)
{
    public bool Carnivoros { get; init; } = carnivoros;

    public override void Comer()
    {
        base.Comer();
        Console.WriteLine($"He comido: {(Carnivoros ? "carne" : "hierba")}");
    }
}

```

```

Mamifero leon = new Mamifero("León", 210, true);
leon.Comer();

```

```

Ñam ñam. Ahora peso 210,49 kilos
He comido: carne

```

Clase Object

En C# existe una clase base principal llamada [Object](#) de la cual todas las clases, incluidas las que nosotros creamos, heredan. Es decir, es la clase base principal de todo el lenguaje. Eso sí, no hace falta especificar esa herencia, ya que es implícita.

La clase `Object` nos ofrece una serie de métodos básicos útiles (y que todas las clases tienen ya que los heredan) que podemos sobrescribir y veremos a continuación.

Método ToString

El método [ToString](#) se utiliza generalmente para devolver una cadena que describa el objeto. A la hora de imprimir por consola un objeto o concatenarlo con una cadena, se llama automáticamente al método `ToString`, por lo que no es necesario llamarlo de forma explícita. Como se hereda de `Object`, si se sobrescribe hay que indicarlo mediante la palabra **override**.

```

public class Animal(...)
{
    ...
}

```

```

    public override string ToString() => $"{Nombre}: {Peso:F2} kilos";
}

```

```

Mamifero leon = new Mamifero("León", 210, true);
Console.WriteLine(leon);

```

León: 210,00 kilos

Método Equals

El método Equals permite comparar si 2 objetos son iguales, por ello recibe otro objeto por parámetro y debemos devolver un booleano indicando si a partir de las propiedades que queremos comprobar (por ejemplo el DNI de una persona), son idénticos.

Para comparar si 2 objetos son iguales debemos utilizar Equals en lugar de ==. Aunque se puede sobrescribir el comportamiento del operador == también (se denomina sobrecarga de operadores), muchos lenguajes no permiten esto y es conveniente tener mecanismos para comparar si 2 objetos tienen las mismas características o si por el contrario son el mismo objeto (referencia), ya que esto último haría que las modificaciones en uno afecten por igual al otro.

Al sobrescribir el método Equals, también se debe sobrescribir el método GetHashCode. Dicho método genera un identificador numérico para el objeto a partir de las propiedades que queramos (generalmente las mismas que compararemos en el método Equals). Para ello nos podemos servir de la clase Tuple, que sirve para almacenar uno o más objetos cualquiera, y nos generará dicho código hash a partir de los elementos que contiene.

```

public class Animal(...)
{
    ...

    public override int GetHashCode() => Tuple.Create(nombre,
peso).GetHashCode();

    public override bool Equals(object obj)
    {
        // Comprobamos que el objeto no es nulo y sea Animal o derivado
        if(obj == null || ! (obj is Animal)) return false;
        Animal a = obj as Animal; // Casting -> Igual que hacer (Animal)obj.
        return nombre == a.Nombre && peso == a.Peso; // mismo nombre y peso
    }
}

```

```

Mamifero leon = new Mamifero("León", 210, true);
Mamifero leon2 = new Mamifero("León", 210, true);
if(leon == leon2)
    Console.WriteLine("Son el mismo objeto (referencia)");
else
    Console.WriteLine("No son el mismo objeto (referencia)");

if(leon.Equals(leon2))
    Console.WriteLine("Son iguales");
else
    Console.WriteLine("No son iguales");

```

No son el mismo objeto (referencia)

Son iguales

Como veremos más adelante, los **records**, por defecto, ya implementan el método Equals y ToString, por lo que no tenemos que implementarlos si nos sirve su comportamiento por defecto.

Clonar objetos

El método **MemberwiseClone** crea una copia superficial del objeto. Es decir, crea un objeto del mismo tipo que el actual, copiando el valor de las propiedades de uno en el otro. Este método no se puede sobrescribir, pero si utilizar para crear nuestros propios métodos de clonado.

```
public class Animal
{
    ...

    public Animal Clone()
    {
        return MemberwiseClone() as Animal;
    }
}

Animal leon = new Animal("León", 210);
Animal leon2 = leon.Clone();
if(leon == leon2)
    System.Console.WriteLine("Son el mismo objeto (referencia)");
else
    System.Console.WriteLine("No son el mismo objeto (referencia)");

if(leon.Equals(leon2))
    System.Console.WriteLine("Son iguales");
else
    System.Console.WriteLine("No son iguales");

No son el mismo objeto (referencia)
Son iguales
```

Hemos mencionado antes que este método crea una copia superficial, por lo que los objetos o arrays que forman parte del objeto como atributos del mismo no se están copiando realmente sino que se están asignado las mismas referencias que en el objeto original (comportamiento por defecto del operador de asignación =).

Si queremos generar una copia completa, en profundidad, debemos también clonar todos los objetos internos del array. Por ejemplo, vamos a clonar un objeto de la clase Zoo que se caracteriza por tener una colección de animales, por lo que clonaremos también dichos animales.

```
class Zoo(string nombre, int capacidad)
{
    private string nombre = nombre;
    private Animal[] animales = new Animal[capacidad];
    public int NumAnimales { get; private set; } = 0;

    public void AddAnimal(Animal a)
    {
        if (NumAnimales < animales.Length)
        {
            animales[NumAnimales++] = a;
        }
    }
}
```



```

        else
        {
            Console.WriteLine("Error: El zoo ha alcanzado su capacidad máxima.");
        }
    }

    public Animal GetAnimal(int pos)
    {
        if (pos >= 0 && pos < animales.Length)
        {
            return animales[pos];
        }

        throw new IndexOutOfRangeException("El animal al que intentas acceder no existe");
    }

    public Zoo Clone()
    {
        var otroZoo = (Zoo)MemberwiseClone();
        // También debemos generar un nuevo array (por defecto es el mismo)
        otroZoo.animales = new Animal[animales.Length];
        for(int i = 0; i < NumAnimales; i++)
        {
            otroZoo.animales[i] = animales[i].Clone();
        }
        return otroZoo;
    }
}

Zoo zoo = new Zoo("Exploración animal", 20);
zoo.AddAnimal(new Animal("Cebra", 190));
Zoo zoo2 = zoo.Clone();
zoo2.GetAnimal(0).SetPeso(999);
Console.WriteLine(zoo.GetAnimal(0));
Console.WriteLine(zoo2.GetAnimal(0));

```

Cebra: 190,00 kilos
Cebra: 999,00 kilos

Clases abstractas

A veces nos interesa que no se puedan instanciar objetos de una clase, sino solamente de sus clases derivadas. En el ejemplo que estamos desarrollando esto sería no permitir crear animales genéricos, sino que deben ser siempre instancias de un tipo específico (Mamífero, Ave, Reptil, ...).

Para impedir que se puedan crear objetos de una clase basta con declararla como abstracta, con la palabra **abstract** delante de la palabra class.

```

public abstract class Animal
{
    ...
}

```

0 references

```

static void Main(s
{
    Zoo zoo = new
    zoo.AddAnimal(new Animal("Cebra", 190));

```

Animal.Animal(string nombre, double peso)
Cannot create an instance of the abstract class or interface 'Animal' (CS0144) [Mi Proyecto]
Quick Fix... Peek Problem

Ahora solamente podemos crear objetos de clases derivadas de Animal:

```
Zoo zoo = new Zoo("Exploración animal", 20);
zoo.AddAnimal(new Mamifero("Cebra", 190, false));
```

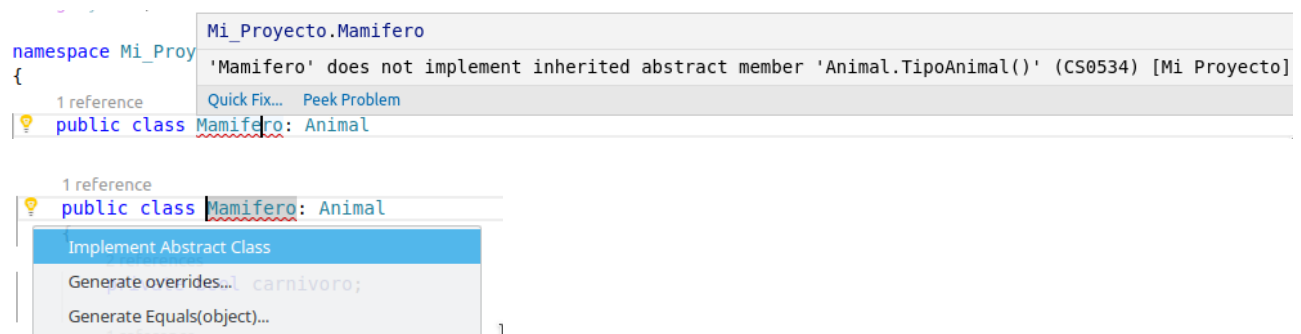
También se pueden declarar métodos abstractos. Esto obliga a 2 cosas: el método no se puede implementar (eso lo deben hacer las clases derivadas de forma **obligatoria**), y la clase debe ser declarada como abstracta también, ya que no tiene sentido crear objetos de una clase que tenga un método que no está implementado. Diferencias entre **abstract** y **virtual** para los métodos:

abstract → El método no puede estar implementado, y **obligatoriamente** las clases derivadas deben implementarlo a su manera (override), o bien declararlo como abstracto (para que otras clases derivadas a su vez de estas lo implementen).

virtual → El método se implementa en la clase base y **opcionalmente**, las clases derivadas pueden sobrescribirlo.

```
public abstract class Animal
{
    ...

    public abstract string TipoAnimal();
}
```



```
public class Mamifero: Animal
{
    ...

    public override string TipoAnimal() => "Mamífero";
}
```

Nota: Una clase derivada o un método sobrescrito (override) también pueden ser declarados como abstractos a su vez, por lo que estaríamos obligando a generar un nuevo nivel de herencia a partir de dicha clase si queremos crear objetos.

Interfaces

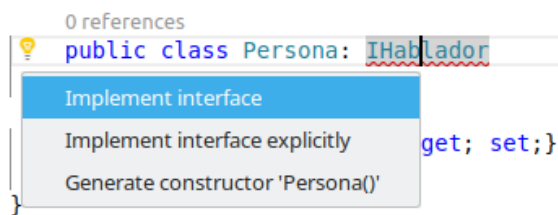
Una interfaz es como una clase completamente abstracta que solo contiene métodos, y en la cual **todos** sus métodos son abstractos. Sin embargo no se utiliza la palabra abstract ya que está implícita. Tampoco se utiliza el modificador public, ya que no pueden haber métodos privados.

Las interfaces se utilizan para definir una serie de comportamientos (métodos) que deben implementar las clases que hereden de dicha interfaz.

Para heredar de una interfaz se hace exactamente lo mismo que para heredar de una clase base. Con la diferencia de que podemos indicar más de una interfaz separándolas con una coma. En resumen, en C# se puede heredar como máximo de 1 clase, y de todas las interfaces que se quiera.

Las interfaces, aunque esto no es obligatorio ni mucho menos, se suelen nombrar con la letra i mayúscula delante del nombre para luego detectar más fácilmente que se trata de una interfaz.

```
public interface IHablador
{
    void Hablar();
}
```



```
public class Persona: IHablador
{
    public string Nombre {get; set;}

    public void Hablar() => Console.WriteLine($"Hola, me llamo {Nombre}");
}
```

```
public abstract class Animal: IHablador
{
    ...

    // Además de implementar el método de IHablador, permite sobrescribirlo a
    // clases derivadas
    public virtual void Hablar() => Console.WriteLine($"{nombre}: (emite un
    extraño sonido...)");
}
```

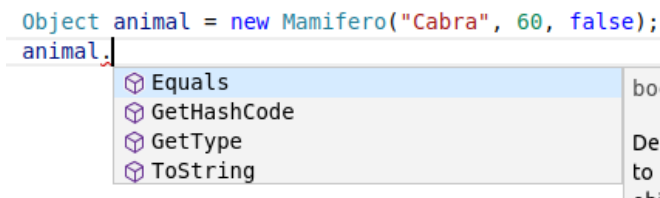
Polimorfismo

El polimorfismo es un concepto amplio en la programación orientada a objetos. Nosotros vamos a ver el concepto más común, que es la posibilidad de agrupar objetos en base a una clase o interfaz de la cual heredan.

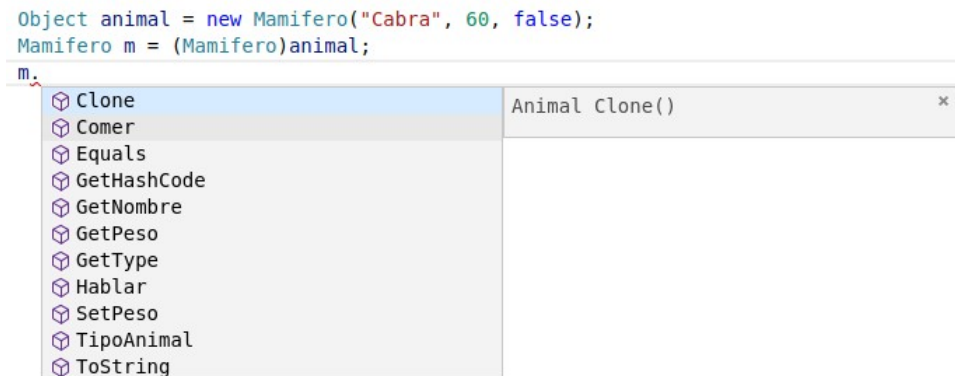
Esto tiene sus ventajas y desventajas. Por ejemplo, hemos visto que todas las clases tienen una antecesora común: la clase Object. Esto quiere decir que podemos declarar cualquier objeto como de tipo Object. De hecho, cuando lo tratamos como un objeto de una clase base o interfaz de la cual se hereda, no hace falta hacer ningún tipo de casting (está implícito).

```
Object animal = new Mamifero("Cabra", 60, false);
```

Aunque internamente en la memoria, está almacenado un objeto de tipo Mamifero con toda su información, le estamos diciendo al compilador, que se trata de un objeto de la clase Object, por lo que sólo podremos acceder a los métodos y propiedades de este (es correcto ya que todos los objetos las heredan).



Para acceder a los métodos del mamífero que sabemos que acabamos de crear, habría que hacer un casting a la clase Mamifero.



Si solamente necesitamos acceder a un método de dicha clase derivada, podemos hacer el casting y la llamada en una sola instrucción.

```
((Mamifero) animal).Hablar();
```

Otra forma de hacer un casting de tipos:

```
(animal as Mamifero).Hablar();
```

Se puede hacer un casting a cualquier clase de la cual derive el objeto. Lo único que hay que recordar es que cuando pasamos de una clase más elevada en la jerarquía a una derivada de la misma, el casting debe ser explícito.

```
Object animal = new Mamifero("Cabra", 60, false);  
(animal as Animal).Comer();
```

Como ya sabemos, si tenemos definido un objeto como que pertenece a una clase base (polimorfismo), solo podemos acceder a los miembros públicos de dicha clase base. Sin embargo, cuando un método está sobrescrito en la clase cuya instancia hemos creado, siempre se ejecutará la implementación de la clase que hayamos utilizado para instanciar el objeto. Es decir, el compilador trata al objeto como si perteneciera a la clase base a la hora de acceder a los métodos, pero internamente sigue siendo un objeto de la clase originalmente utilizada para crearlo.

```
Object animal = new Mamifero("Cabra", 60, false);  
Animal cabra = animal as Animal;  
cabra.Comer();  
System.Console.WriteLine(cabra.TipoAnimal());
```

```
Ñam ñam. Ahora peso 60,44 kilos  
He comido: hierba  
Mamífero
```

El polimorfismo es muy útil para agrupar objetos del mismo tipo en colecciones. Por ejemplo, en un array de tipo Object, podríamos meter cualquier cosa (todo hereda de dicha clase), en un array de tipo Animal, podríamos meter objetos del tipo Mamífero, Ave, etc. O incluso en un array del tipo interfaz IHablador, podríamos meter cualquier objeto de una clase que implemente dicha interfaz (directa o indirectamente).

```
Object[] cosas = new Object[3];
cosas[0] = 234;
cosas[1] = "Hola";
cosas[2] = new Mamifero("Perro", 45, true);

foreach(Object cosa in cosas)
{
    System.Console.WriteLine(cosa);
}

234
Hola
Perro: 45,00 kilos
```

En el ejemplo del Zoo vimos como teniendo un array de objetos del tipo Animal podíamos incluir todo tipo de animales dentro.

```
class Zoo(string nombre, int capacidad)
{
    private string nombre = nombre;
    private Animal[] animales = new Animal[capacidad];
    public int NumAnimales { get; private set; } = 0;

    public Zoo(string nombre, int capacidad) ...

    public void AddAnimal(Animal a) ...

    public Animal GetAnimal(int pos) ...
    ...
}

Zoo zoo = new Zoo("Exploración animal", 20);
zoo.AddAnimal(new Mamifero("Cebra", 190, false));
zoo.AddAnimal(new Ave("Loro", 0.9, false));
```

Si queremos comprobar previamente si un objeto pertenece a una clase (da igual si es la clase del objeto o una clase base de la cual hereda) para hacer un casting y poder acceder a dichos métodos, se puede utilizar la condición **objeto is Clase**.

```
Animal a = zoo.GetAnimal(0);
if (a is Mamifero)
{
    Mamifero m = a as Mamifero;
    System.Console.WriteLine($"{m.GetNombre()} es un mamífero
{ (m.EsCarnivoro() ? "carnívoro" : "herbívoro")}");
}
```

Otra forma más moderna de comprobar el tipo de un objeto es utilizando la estructura switch con **Pattern Matching**:

```
// Switch clásico
switch (zoo.animales[i])
{
    case Mamifero mamifero:
        //...
        break;
    case Ave ave:
```

```

        //...
        break;
    default:
        // El caso por defecto obligatorio para mantener el código seguro
        throw new InvalidOperationException("Tipo de animal no soportado");
    }

// Switch que devuelve resultado
string resultado = a switch
{
    Mamifero mam when mam.GetPeso() > 100 => $"Mamífero grande: {mam.GetNombre()}",
    Mamifero mam => $"Mamífero normal: {mam.GetNombre()}",
    Ave ave => "Es un ave",
    null => "El objeto es nulo",
    _ => "Cualquier otro tipo de animal" // Caso por defecto (Discard)
};

```

Igual que hablamos de clases base, también se puede utilizar una interfaz común que implementen los objetos para hacer polimorfismo. En este caso solo podremos acceder a los métodos definidos en dicha interfaz.

```

// Mamifero hereda de Animal, que implementa IHablador
IHablador hablador = new Mamifero("Rinoceronte", 1000, true);
IHablador hablador2 = new Persona("Pepito");

hablador.Hablar();
    hablador2.Hablar();
}
}

```

Herencia con records

La herencia funciona exactamente igual que en las clases, con una única regla estricta: un **record** solo puede heredar de otro record (no puedes mezclar una class tradicional con un record en una línea de herencia).

Aprovechando los constructores primarios, la sintaxis para definir herencia es muy limpia, ya que los parámetros se pasan directamente al registro base en la propia firma:

```
// Registro Base
public record Animal(string Nombre, double Peso);

// Registro Derivado (Pasa Nombre y Peso al constructor base)
public record Mamifero(string Nombre, double Peso, bool Carnivoro)
: AnimalRecord(Nombre, Peso);
```

Métodos de la clase Object autogenerados

Cuando se define un record automáticamente se generan los siguientes métodos heredados de object:

- **ToString()** personalizado: Ya no imprime el nombre del espacio de nombres. Devuelve una cadena perfectamente formateada con el nombre del registro y los valores actuales de todas sus propiedades.
- **Equals()** y **GetHashCode()** por valor: En una clase normal, Equals evalúa si los objetos apuntan a la misma dirección de memoria (referencia). En un record, Equals se reescribe automáticamente para realizar una comparación por valor. Dos registros independientes en memoria serán considerados iguales (true) si todas sus propiedades contienen los mismos datos.

También se sobrecarga el operador == para comparar objetos de la misma manera que utilizando Equals.

Clonación Inmutable (La expresión with)

En el caso de los records, no es necesario generar un constructor de copia siempre que las propiedades que componen el objeto sean a su vez inmutables (internamente).

```
var leon1 = new MamiferoRecord("León", 210, true);

// Clonación modificando solo el Peso
var leon2 = leon1 with { Peso = 215 };

// 3. Demostración del ToString() automático
System.Console.WriteLine(leon1); // Mamifero { Nombre = León, Peso = 210, Carnivoro = True }

System.Console.WriteLine(leon2); // Mamifero { Nombre = León, Peso = 215, Carnivoro = True }

var leonClonIdentico = leon1 with { }; // Clon exacto sin cambios
System.Console.WriteLine(leon1 == leonClonIdentico); // True (Son objetos distintos en memoria, pero sus valores son idénticos)
```